Clean CPU evidence: 0.6.0-registry-aa7347a

Measured source `aa7347a201ce9294b02990b7598af146470bca65`, tree `a2cf4ff5e2bc34249b516316030eb7683d6f3647`; dirty before/after: false. Library source: registry 0.6.0. One pinned CPU; f64 operations and explicitly named f32 constructors; row-major inputs.

Unavailable historical Tape stages: training_forward_2x4x8x4_f64, training_record_2x4x8x4_f64, training_backward_2x4x8x4_f64, training_sgd_2x4x8x4_f64. registry Tape lacks batched matmul/layer_norm; no substitute derivatives used.

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
cholesky_factor_64_f64/faer	7.940 / 9.294	8.664 / 8.828	2	33280	4.06
cholesky_factor_64_f64/kernix	22.908 / 29.241	24.888 / 32.092	2	65536	4.10
cholesky_factor_64_f64/nalgebra	13.251 / 14.817	14.382 / 15.043	1	32768	3.89
cholesky_factor_64_f64/numpy	10.996 / 14.775	11.205 / 11.801	unavailable	unavailable	33.07
create_ones_64x64_f32/candle	0.218 / 0.224	0.243 / 0.257	7	16696	4.43
create_ones_64x64_f32/faer	0.190 / 0.197	0.177 / 0.192	1	16384	3.98
create_ones_64x64_f32/kernix	0.203 / 0.206	0.197 / 0.271	1	16384	3.87
create_ones_64x64_f32/ndarray	0.155 / 0.158	0.151 / 0.168	1	16384	3.75
create_ones_64x64_f32/numpy	1.317 / 1.402	1.377 / 1.411	unavailable	unavailable	32.72
create_zeros_64x64_f32/candle	0.152 / 0.168	0.158 / 0.168	5	16648	4.29
create_zeros_64x64_f32/faer	0.173 / 0.183	0.172 / 0.187	1	16384	4.07
create_zeros_64x64_f32/kernix	0.146 / 0.148	0.143 / 0.150	1	16384	3.75
create_zeros_64x64_f32/ndarray	0.098 / 0.100	0.095 / 0.101	1	16384	3.76
create_zeros_64x64_f32/numpy	0.417 / 0.480	0.420 / 0.473	unavailable	unavailable	32.68
matmul_128_f64/candle	74.073 / 81.753	76.093 / 91.269	9	131352	5.44
matmul_128_f64/faer	83.526 / 91.993	84.245 / 103.989	1	131072	4.59
matmul_128_f64/kernix	114.500 / 127.866	120.328 / 127.049	3	327680	4.48
matmul_128_f64/nalgebra	116.693 / 135.646	116.903 / 125.452	2	327680	4.02
matmul_128_f64/ndarray	112.758 / 127.762	118.630 / 124.072	2	327680	4.36
matmul_128_f64/numpy	76.146 / 84.889	79.440 / 80.989	unavailable	unavailable	34.35
matmul_32_f64/candle	1.499 / 1.679	1.599 / 1.621	9	8592	4.91
matmul_32_f64/faer	1.628 / 1.697	1.674 / 1.758	1	8192	4.19
matmul_32_f64/kernix	1.639 / 1.887	1.679 / 1.774	3	24576	4.10
matmul_32_f64/nalgebra	1.423 / 1.438	1.490 / 1.510	2	24576	3.80
matmul_32_f64/ndarray	1.486 / 1.656	1.523 / 1.598	2	24576	3.74
matmul_32_f64/numpy	2.802 / 4.040	2.671 / 2.836	unavailable	unavailable	33.00
matmul_batched_4x16x32x16_f64/candle	2.120 / 2.610	2.159 / 2.283	25	25504	4.78
matmul_batched_4x16x32x16_f64/kernix	284.196 / 296.708	279.639 / 281.707	5	8248	3.98
matmul_batched_4x16x32x16_f64/numpy	3.250 / 3.596	3.142 / 3.318	unavailable	unavailable	32.98
matmul_sum_backward_32x16x8_4/candle	4.206 / 5.291	4.350 / 5.908	113	27988	5.53
matmul_sum_backward_32x16x8_4/kernix	3.484 / 3.709	3.113 / 3.207	19	18376	4.05
qr_256x64_f64/faer	406.409 / 505.275	416.610 / 537.323	7	331776	4.80
qr_256x64_f64/kernix	503.391 / 602.436	514.579 / 532.231	6	557568	4.79
qr_256x64_f64/nalgebra	377.013 / 418.281	379.203 / 390.702	4	295424	4.02
qr_256x64_f64/numpy	351.733 / 389.831	353.047 / 369.211	unavailable	unavailable	33.99

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
qr_96x32_f64/faer	66.109 / 77.064	68.066 / 72.332	7	67584	4.58
qr_96x32_f64/kernix	41.846 / 50.447	44.696 / 50.108	6	106752	4.11
qr_96x32_f64/nalgebra	37.811 / 43.158	37.945 / 39.164	4	57600	3.77
qr_96x32_f64/numpy	40.694 / 47.402	42.222 / 44.367	unavailable	unavailable	33.28
softmax_256x64_f64/candle	79.719 / 88.244	80.749 / 89.958	37	265200	5.62
softmax_256x64_f64/kernix	88.736 / 117.349	90.820 / 103.291	1	131072	4.40
softmax_256x64_f64/ndarray	71.720 / 80.000	74.258 / 79.235	1	131072	4.23
softmax_256x64_f64/numpy	103.231 / 114.441	103.234 / 106.104	unavailable	unavailable	33.11

Method and limits

- Per-call quantiles use nearest ranks over 256 individual warmed calls; batch quantiles use 15 means over separate 10-ms batches. They are separate distributions. Raw samples remain in raw.json.
- Eight warmup calls follow validation. Timing includes allocation, dispatch, computation and output destruction. The first call field excludes output destruction. Inputs and library/runtime imports are prepared outside timing.
- Three extra processes record their first operation calls. These are first-use observations, not cache-flushed or device-startup latency. Timer-pair overhead is reported without subtraction; tiny calls are clock-sensitive.
- ndarray and Candle softmax are explicit stable compositions, not claims about a native softmax API.
- Constructor cases allocate owned 64x64 f32 tensors. Pre-0.6 registry baselines use full(value), explicitly marked in metadata; current workspace runs use zeros/ones. No nested literal reinterpretation is involved.
- All GEMM inputs are C-order and match elementwise. Native output layout differs by library. QR materializes thin Q and R in every implementation. Cholesky returns reusable factors (or NumPy L); no extra factor materialization is timed.
- Kernix checks finite values/symmetry. Some competitors validate a narrower contract. No finite-data precondition check was removed to improve timing. Accuracy results and budgets are retained per worker.
- The backward case uses an actual reverse-mode graph in Kernix and Candle for sum(A @ W), with A constant and W trainable. Both copy input/weight data and returned gradients; no manual derivative is used as a framework ranking.
- Allocations are measured in a separately compiled instrumented binary after warmup, outside all timing loops. Counts cover Rust global-allocator requests, not native allocator internals, stack memory or all NumPy allocations. NumPy allocation counts are unavailable, not zero.
- Requested-byte peak is additional live Rust allocation during one call, including output; it is not scratch-only memory or peak RSS. RSS is the process high-water mark including fixtures, validation and runtime, read from Linux VmHWM. Other platforms may report unavailable.
- The published cases are allocation-inclusive CPU APIs and graph workloads. Pure internal kernel-only and end-to-end application/model latency are not measured. No workload result implies a complete application frame rate.
- faer is explicitly serial; Rayon and BLAS thread counts are one. No GPU comparison. Python/NumPy timings include Python dispatch. Backend and dependency versions/configuration, governor and RAM are recorded.
- Training stages use batched matmul + bias + layer_norm + tanh + step-two feature slice + MSE (Candle index_select). Forward excludes graph recording; record includes parameter/input/target snapshots and eager forward/loss; backward reuses a prepared graph and returns fresh derivatives. Allocation probes measure the corresponding stage, including graph values/scratch, not graph-container-only bytes. SGD uses fixed precomputed real gradients and includes transactional validation/parameter updates; it is not an end-to-end converging training loop. Candle normalization is an explicit population-statistics composition; its checks/rounding differ. Historical registry baselines without these Tape operations omit the stages, recorded in metadata, rather than substitute manual derivatives.
- A shared laptop has uncontrolled load/frequency/cache effects. Report comparator wins and losses; no universal ranking follows. The source archive comes from committed bytes, with result directories excluded. Library dependencies are pinned by the saved driver lockfile.